

UNIVERSITY OF TRANSPORT HO CHI MINH CITY

INFORMATION SECURITY

MATHEMATICAL FOUNDATIONS

PhD. Tran The Vinh

ALGORITHMS AND COMPUTATIONAL COMPLEXITY

Concept of Algorithms

In mathematics and computer science, an algorithm is a finite sequence of well-defined and logically ordered computational steps designed to solve a specific problem or perform a particular task.

Algorithms serve as the foundation of all computational and data-processing operations in computer systems. An algorithm accepts input data, performs a sequence of deterministic operations, and produces corresponding output results.

Fundamentally, an algorithm must precisely specify:

- input data;
- processing rules;
- execution sequence;
- termination conditions;
- output results.

In many mathematical and computational problems, algorithms are commonly implemented using iterative or recursive procedures to optimize problem-solving processes.

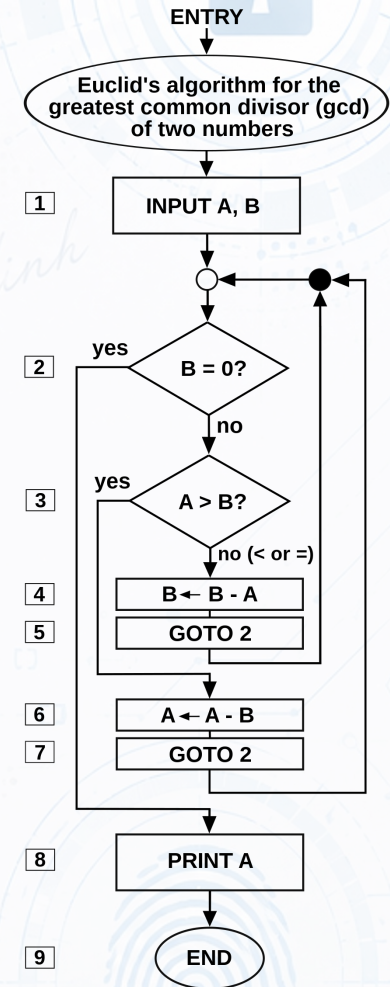
Representation of Algorithms

Algorithms may be represented using different forms depending on the purpose of analysis, design, or implementation, including:

- natural language descriptions;
- pseudocode;
- flowcharts;
- DRAKON diagrams;
- programming languages;
- logical models or block diagrams.

In practice, pseudocode and flowcharts are among the most widely used representations because they provide intuitive descriptions of algorithmic logic independent of specific programming languages.

Algorithms may range from very simple to highly complex depending on the scale and nature of the problem being addressed.



ALGORITHMS AND COMPUTATIONAL COMPLEXITY

Fundamental Properties of Algorithms

A well-designed algorithm should satisfy the following essential properties:

- **Finiteness:** An algorithm must terminate after a finite number of execution steps. An algorithm that never halts is considered invalid.
- **Definiteness:** Every step of the algorithm must be precisely and unambiguously defined so that identical inputs always produce identical outputs.
- **Correctness:** The algorithm must generate correct results for all valid input cases.
- **Effectiveness:** Each operation in the algorithm must be sufficiently basic and feasible to execute within finite computational resources.
- **Generality:** The algorithm should solve an entire class of problems rather than only a specific instance.

Algorithm Complexity

Concept: Algorithm complexity refers to the amount of computational resources required to execute an algorithm.

The two most important computational resources are:

- execution time (*time complexity*);
- memory consumption (*space complexity*).

Complexity typically depends on the input size n . As the input size increases, the number of required computational operations also increases.

Therefore, algorithm complexity is commonly represented as a function of the input size:

$$T(n)$$

where:

- n denotes the size of the input data;
- $T(n)$ denotes the number of computational steps or execution time required by the algorithm.

Big-O Complexity Notation

In computer science, algorithm complexity is commonly expressed using Big-O notation, which describes the asymptotic growth rate of computational resources as the input size increases.

Examples include:

- $O(1)$: constant complexity;
- $O(\log n)$: logarithmic complexity;
- $O(n)$: linear complexity;
- $O(n^2)$: quadratic complexity;
- $O(2^n)$: exponential complexity;
- $O(n!)$: factorial complexity.

In cryptography, complexity analysis is particularly important because the security of many cryptographic systems depends directly on the computational effort required to break them.

Bit Complexity

In computer systems, all data is represented in binary form using two logical states: 0 and 1.

A binary digit is called a bit.

If a value is represented using k binary digits, it is referred to as a k -bit value.

In cryptography and large-number computation, algorithm complexity is often evaluated in terms of the number of bit operations because this provides a more accurate measurement of actual computational cost.

Bit operations include: AND; OR; XOR; NOT; bit shifting; arithmetic operations on binary data.

For example, the computational costs of modern cryptographic algorithms such as RSA, AES, and SHA are typically analyzed based on the number of required bit operations.

LOGICAL OPERATORS

Concept

Exclusive OR, commonly referred to as **XOR** or **addition modulo 2**, is one of the fundamental logical operators widely used in cryptography, coding theory, digital signal processing, and logic circuit design.

From a mathematical perspective, logical operators are based on **Boolean Algebra**, developed by the English mathematician **George Boole** in the nineteenth century. Boolean algebra later became a fundamental theoretical basis for the design of electronic computers, digital circuits, processors, and microelectronic systems.

In Boolean algebra, a two-variable logical operator can be represented as:

$$f = f(x, y)$$

where x and y are input variables, and f is the output variable. All of these variables are **logical variables**, meaning that they can take only one of two values from the set:

$$\{0, 1\}$$

where:

- 0 represents **False**;
- 1 represents **True**.

The XOR operation has the following property: its output is 1 when the two inputs are different, and 0 when the two inputs are identical. Therefore, XOR is frequently used in cryptography to combine plaintext with a key, particularly in stream ciphers and One-Time Pad encryption.



LOGICAL OPERATORS

NOT

The negation function of the input variable a is denoted by $\bar{a}$ (or $NOT\ a$).

The value of the output variable $\bar{a}$ is always the logical opposite of the input variable a .

In other words, $\bar{a}$ always represents the logical complement of a .

a	$\bar{a}$
0	1
1	0

AND

The logical conjunction function is represented as:

$$f = a \wedge b$$

(or equivalently $f = a \cap b$).

The output variable f takes the value 1 if and only if both input variables simultaneously have the value 1. In all other cases, f takes the value 0.

Meaning:

f is true if and only if both a and b are true simultaneously.

The AND operator is one of the fundamental logical operations widely used in Boolean algebra, digital circuit design, computer architecture, and cryptographic computation.

a	b	$a \wedge b$
0	0	0
0	1	0
1	0	0
1	1	1

LOGICAL OPERATORS

OR

The logical disjunction function is represented as:

$$f = a \vee b$$

(or equivalently $f = a \cup b$).

The output variable f takes the value 1 if at least one of the input variables a or b has the value 1, including the case where both a and b are equal to 1.

Meaning:

f is true if a is true, or b is true, or both a and b are true simultaneously.

The OR operator is a fundamental Boolean operation widely used in digital logic design, conditional computation, and logical decision-making systems.

a	b	$a \vee b$
0	0	0
0	1	1
1	0	1
1	1	1

XOR

Exclusive OR (XOR), also known as **addition modulo 2**, is represented as:

$$f = a \oplus b$$

or:

$$f = a \text{ XOR } b$$

The output variable f takes the value 1 if and only if exactly one of the two input variables a or b has the value 1.

Meaning:

if both a and b are false (0,0) or both are true (1,1), then $f = 0$;

if only one of the two variables is true (1,0 or 0,1), then $f = 1$.

The XOR operator plays a particularly important role in cryptography because it is reversible and computationally efficient. It is widely used in stream ciphers, block ciphers, hash functions, error-detection codes, and One-Time Pad encryption systems.

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

XOR Function in Cryptography

The XOR (*Exclusive OR*) operator plays a critically important role in cryptography due to its logical properties and its simple yet effective reversibility.

Logical Meaning of XOR

The XOR operator represents the concept of “difference” between two input variables:

- if the two input variables have identical values, the XOR result is 0;
- if the two input variables have different values, the XOR result is 1.

The expression:

$$a \oplus b$$

evaluates to true if and only if the two inputs are different.

Applications of XOR in Encryption

In many symmetric cryptographic systems, the XOR operation is used to combine plaintext with a secret key.

Let:

P : plaintext;

K : secret key;

C : ciphertext.

The encryption process is defined as:

$$C = P \oplus K$$

Because XOR possesses the self-inverse property, decryption can be performed by XORing the ciphertext with the same key:

$$P = C \oplus K$$

The fundamental property is:

$$(P \oplus K) \oplus K = P$$

because:

$$K \oplus K = 0$$

and:

$$P \oplus 0 = P$$

XOR Function in Cryptography

Example

Encryption

$$\begin{aligned}P &= 100110001 \\K &= 001111001 \\C &= P \oplus K = 101001000\end{aligned}$$

Decryption

$$\begin{aligned}P &= C \oplus K \\101001000 \oplus 001111001 &= 100110001\end{aligned}$$

The original plaintext is successfully recovered.

Remarks

1. Impact of Key Length

The length of the secret key directly affects the security level of a symmetric cryptosystem.

If a key has length n bits, the total number of possible keys is: 2^n

For example:

a 56-bit key provides: 2^{56} possible key combinations;

a 128-bit key provides: 2^{128} possible key combinations.

The larger the key space, the more computationally infeasible a brute-force attack becomes.

2. High Computational Efficiency

The XOR operation is an extremely simple hardware-level logical operation and therefore can be executed very efficiently.

As a result, XOR is widely used in:

- stream ciphers;
- block ciphers;
- hash functions;
- pseudo-random generators;
- One-Time Pad;
- AES;
- DES;
- ChaCha20.

However, XOR alone is insufficient to construct a secure cryptographic system unless it is combined with appropriate diffusion and confusion mechanisms.

PRIME NUMBERS

PRIME NUMBERS

The symbol $\mathbb{Z}$ denotes the set of integers, while $\mathbb{N}$ denotes the set of natural numbers.

For two integers $a, b \in \mathbb{Z}$, we say that a divides b if there exists an integer k such that:

$$b = a \cdot k$$

In this case, a is called a divisor (or factor) of b , denoted by: $a \mid b$ and read as “ a divides b ”.

Divisibility possesses several important properties widely used in number theory and cryptography. For integers:

$$a, b, c \in \mathbb{Z}$$

if $a \mid b$ then $a \mid bc$ meaning that if a divides b , then a also divides the product bc .

If $a \mid b$ and $b \mid c$ then $a \mid c$

which represents the transitive property of divisibility.

If $a \mid b$ and $a \mid c$ then $a \mid (b \pm c)$

meaning that a also divides the sum or difference of b and c .

Conversely, if $a \mid b$ but $a \nmid c$ then $a \nmid (b \pm c)$

A natural number greater than 1 is called a prime number if it has exactly two positive divisors: 1 and itself. In other words, a prime number is divisible only by 1 and itself.

Examples of prime numbers include: 2, 3, 5, 7, 11, 13

An important consequence in number theory states that if p is a prime number and: $p \mid ab$

then at least one of the integers a or b must be divisible by p : $p \mid a$ or $p \mid b$

This property serves as a fundamental basis for many modern cryptographic algorithms, including RSA and public-key cryptosystems.

The greatest common divisor (*Greatest Common Divisor* – GCD) of two natural numbers a and b , denoted by: $\gcd(a, b)$

is the largest positive integer that divides both a and b .

For example:

$$\gcd(12, 18) = 6$$

Two natural numbers a and b are called coprime (or relatively prime) if:

$$\gcd(a, b) = 1$$

This means that the two integers share no positive common divisor other than 1. The concept of coprime integers plays a crucial role in modular arithmetic, the Euclidean algorithm, congruence theory, and modern cryptography.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

EUCLIDEAN ALGORITHM

The Euclidean Algorithm, also known as Euclid's Algorithm, is an efficient method for computing the greatest common divisor (GCD) of two integers. This algorithm has been known since approximately 300 BCE. The ancient Greek mathematician Euclid described the algorithm in his famous work *Elements*.

Example

Compute the greatest common divisor of 36 and 129.

First, divide 129 (the larger integer) by 36:

$$129 = 36 \times 3 + 21$$

The pair (36, 21) is then used in the next iteration.

$$36 = 21 \times 1 + 15$$

$$21 = 15 \times 1 + 6$$

$$15 = 6 \times 2 + 3$$

$$6 = 3 \times 2$$

Finally, we obtain:

$$3 = \gcd(6, 3) = \gcd(15, 6) = \gcd(21, 15) = \gcd(36, 21) = \gcd(129, 36).$$

Thus:

$$\gcd(129, 36).$$

Lemma

Suppose:

$$a = bq + r$$

where a, b, q, r are integers. Then:

$$\gcd(a, b) = \begin{cases} b & \text{nếu } r = 0 \\ \gcd(b, r) & \text{nếu } r \neq 0 \end{cases}$$

This lemma forms the mathematical foundation of the Euclidean Algorithm and explains why the repeated division process eventually produces the greatest common divisor.

EXTENDED EUCLIDEAN ALGORITHM

The Extended Euclidean Algorithm is used to solve linear Diophantine equations of the form:

$$a \cdot x + b \cdot y = c$$

where a , b , and c are integer coefficients, while x and y are integer variables.

A necessary and sufficient condition for this equation to have integer solutions is that:

$$\gcd(a, b)$$

must divide c .

In other words:

$$\gcd(a, b) \mid c$$

This statement is based on the following important theorem in number theory:

If:

$$d = \gcd(a, b)$$

then there exist integers x and y such that:

$$a \cdot x + b \cdot y = d$$

This identity is known as **Bézout's Identity** and forms the mathematical foundation of the Extended Euclidean Algorithm.

The Extended Euclidean Algorithm not only computes the greatest common divisor of two integers, but also determines integer coefficients x and y satisfying Bézout's equation. This algorithm plays a fundamental role in number theory, modular arithmetic, and modern cryptography, particularly in applications such as:

- modular inverse computation;
- RSA cryptography;
- public-key cryptosystems;
- congruence equations;
- finite field arithmetic.

EXTENDED EUCLIDEAN ALGORITHM

The Extended Euclidean Algorithm is an extension of the classical Euclidean Algorithm. While the standard Euclidean Algorithm is used only to compute:

$$\gcd(a, b)$$

the Extended Euclidean Algorithm not only determines the greatest common divisor, but also finds two integers x and y such that:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

This result is known as Bézout's Identity and plays a fundamental role in number theory and modern cryptography, particularly in RSA, modular inverse computation, and public-key cryptosystems.

Fundamental Idea of the Algorithm

Suppose two natural numbers are given such that:

$$a > b > 0$$

Define:

$$r_0 = a, \quad r_1 = b$$

The Euclidean division process is then repeatedly applied:

$$\begin{aligned} r_0 &= q_1 r_1 + r_2, & 0 < r_2 < r_1 \\ r_1 &= q_2 r_2 + r_3, & 0 < r_3 < r_2 \\ &\dots \end{aligned}$$

$$\begin{aligned} r_{m-1} &= q_m r_m + r_{m+1}, & 0 < r_{m+1} < r_m \\ r_m &= q_{m+1} r_{m+1} \end{aligned}$$

where:

- q_i denotes the quotient;
- r_i denotes the remainder.

Since the sequence of remainders:

$$r_0 > r_1 > r_2 > \dots$$

is strictly decreasing and always nonnegative, a remainder equal to zero must eventually appear after a finite number of steps.

The last nonzero remainder:

$$r_{m+1}$$

is therefore equal to:

$$\gcd(a, b)$$

EXTENDED EUCLIDEAN ALGORITHM

Construction of the Recursive Relations

The algorithm constructs coefficients:

$$x_i, y_i$$

satisfying:

$$a \cdot x_i + b \cdot y_i = r_i$$

for every i .

Initially:

$$a \cdot 1 + b \cdot 0 = a = r_0$$

and:

$$a \cdot 0 + b \cdot 1 = b = r_1$$

thus:

$$\begin{aligned} x_0 &= 1, y_0 = 0 \\ x_1 &= 0, y_1 = 1 \end{aligned}$$

Assume that:

$$a \cdot x_i + b \cdot y_i = r_i$$

and:

$$a \cdot x_{i+1} + b \cdot y_{i+1} = r_{i+1}$$

From the Euclidean division equation:

$$r_i = q_{i+1}r_{i+1} + r_{i+2}$$

we obtain:

$$r_{i+2} = r_i - q_{i+1}r_{i+1}$$

Substituting the linear representations of r_i and r_{i+1} :

$$r_{i+2} = (a \cdot x_i + b \cdot y_i) - q_{i+1}(a \cdot x_{i+1} + b \cdot y_{i+1})$$

Expanding the expression gives:

$$r_{i+2} = a(x_i - q_{i+1}x_{i+1}) + b(y_i - q_{i+1}y_{i+1})$$

Therefore, the recursive formulas become:

$$x_{i+2} = x_i - q_{i+1}x_{i+1}$$

$$y_{i+2} = y_i - q_{i+1}y_{i+1}$$

Continuing this process until the final iteration produces:

$$x_{m+1}, y_{m+1}$$

such that:

$$a \cdot x_{m+1} + b \cdot y_{m+1} = \gcd(a, b)$$

Importance in Cryptography

The Extended Euclidean Algorithm is extremely important in modern cryptography because it enables the computation of:

- modular inverses;
- RSA private keys;
- solutions to congruence equations;
- arithmetic operations over finite fields.

For example, in RSA cryptography, determining the private decryption key d requires solving:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

This problem is fundamentally equivalent to finding a modular inverse using the Extended Euclidean Algorithm.

EXTENDED EUCLIDEAN ALGORITHM

The Extended Euclidean Algorithm computes the greatest common divisor of two integers a and b , and also determines two integers x and y satisfying Bézout's identity:

$$ax + by = \gcd(a, b)$$

Extended Euclidean Algorithm

Input: Two integers a and b

Output: $d = \gcd(a, b)$, and integers x, y such that $ax + by = d$

Procedure Extended_Euclidean(a, b)

```
old_r := a
r := b
old_x := 1
x := 0
old_y := 0
y := 1
while r ≠ 0 do
  q := old_r div r
  temp_r := old_r - q * r
  old_r := r
  r := temp_r
  temp_x := old_x - q * x
  old_x := x
  x := temp_x
  temp_y := old_y - q * y
  old_y := y
  y := temp_y
end while
```

return $d := \text{old_r}, x := \text{old_x}, y := \text{old_y}$

End Procedure

EXTENDED EUCLIDEAN ALGORITHM

Example

Suppose:

$$a = 29, b = 8$$

The algorithm proceeds through the following steps.

Bước i	r_i	r_{i+1}	r_{i+2}	q_{i+1}	x_i	x_{i+1}	x_{i+2}	y_i	y_{i+1}	y_{i+2}
0	29	8	5	3	1	0	1	0	1	-3
1	8	5	3	1	0	1	-1	1	-3	4
2	5	3	2	1	1	-1	2	-3	4	-7
3	3	2	1	1	-1	2	-3	4	-7	11
4	2	1	0	2						

The algorithm simultaneously produces the results:

$$d = \gcd(29, 8) = 1$$

and:

$$x = -3, y = 11$$

It is easy to verify Bézout's identity:

$$29 \times (-3) + 8 \times 11 = 1$$

Thus, the integers -3 and 11 form a valid solution to the equation:

$$29x + 8y = 1$$

This example demonstrates that the Extended Euclidean Algorithm not only computes the greatest common divisor, but also determines the integer coefficients satisfying Bézout's equation.

MODULO FUNCTION – CONGRUENCE RELATION

The modulo function can be understood simply as the remainder obtained in integer division. To compute X modulo Y (commonly denoted as $X \bmod Y$), we divide X by Y and determine the remainder of that division. Equivalently, we subtract from X the largest multiple of Y that is less than or equal to X . Clearly:

can only take values from:

$$X \bmod Y$$

$$0, 1, \dots, Y - 1$$

Example:

$$25 \bmod 5 = 0$$

In number theory, two integers a and b are said to be congruent modulo n if they have the same remainder when divided by n . This relation is denoted by:

$$a \equiv b \pmod{n}$$

and read as:

“ a is congruent to b modulo n ”.

Such an expression is called a congruence relation (or congruence equation).

Example:

$$18 \equiv 4 \pmod{7}$$

because both 18 and 4 leave the same remainder when divided by 7.

Some fundamental properties of congruence arithmetic are as follows:

- $a \equiv a \pmod{n}$;
- If $a \equiv b \pmod{n}$ then $b \equiv a \pmod{n}$;
- If $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$ then $a \equiv c \pmod{n}$;
- If $a \equiv b \pmod{n}$, $c \equiv d \pmod{n}$ then

$$a \pm c \equiv b \pm d \pmod{n}, ac \equiv bd \pmod{n};$$

These properties form the mathematical foundation of modular arithmetic, which is extensively used in number theory, cryptography, computer security, and public-key cryptographic systems such as RSA.

MODULAR INVERSE IN CONGRUENCE ARITHMETIC

In modular arithmetic, addition, subtraction, and multiplication are always defined over the set:

$$\mathbb{Z}/n\mathbb{Z} = \{0, 1, 2, \dots, n - 1\}$$

However, division is not always possible. To perform division modulo n , the concept of a modular inverse is required. An element a is said to have a modular inverse modulo n if there exists an integer a^{-1} such that:

$$a \cdot a^{-1} \equiv 1 \pmod{n}$$

$$\gcd(a, n) = 1$$

A modular inverse exists if and only if:

The set of invertible elements modulo n is denoted by:

and contains:

$$\varphi(n)$$

elements.

$$(\mathbb{Z}/n\mathbb{Z})^*$$

If $a \equiv b \pmod{n}$ and $c \equiv d \pmod{n}$

with: $\gcd(c, n) = 1$

then:

$$ac^{-1} \equiv bd^{-1} \pmod{n}$$

Example

For:

$$\mathbb{Z}/9\mathbb{Z} = \{0, 1, 2, \dots, 8\}$$

we have:

$$(\mathbb{Z}/9\mathbb{Z})^* = \{1, 2, 4, 5, 7, 8\}$$

Since:

$$5 \times 2 = 10 \equiv 1 \pmod{9}$$

it follows that:

$$5^{-1} \equiv 2 \pmod{9}$$

Therefore:

$$\frac{2}{5} \equiv 2 \cdot 5^{-1} = 2 \cdot 2 = 4 \pmod{9}$$

because:

$$5 \times 4 = 20 \equiv 2 \pmod{9}$$

LINEAR CONGRUENCE EQUATIONS

Consider the linear congruence equation:

$$ax \equiv b \pmod{m}$$

where:

a, b, m are integers;

$m > 0$;

x is the unknown variable.

The solvability of this equation depends on:

$$g = \gcd(a, m)$$

Case 2: $\gcd(a, m) = g > 1$

Two situations may occur.

The congruence has solutions

The congruence has solutions if and only if:

$$g \mid b$$

that is, g divides b .

In this case, dividing the entire congruence by g gives the equivalent congruence:

$$\frac{a}{g}x \equiv \frac{b}{g} \pmod{\frac{m}{g}}$$

Since:

$$\gcd\left(\frac{a}{g}, \frac{m}{g}\right) = 1$$

the reduced congruence always has solutions and can be solved using a modular inverse.

In this case, there exist exactly:

$$g$$

distinct solutions modulo m .

Case 1: $\gcd(a, m) = 1$

When a and m are relatively prime, a possesses a modular inverse:

$$a^{-1} \pmod{m}$$

Multiplying both sides of the congruence by a^{-1} yields the unique solution modulo m :

$$x \equiv a^{-1}b \pmod{m}$$

This is the most fundamental case in modular arithmetic and cryptography.

The congruence has no solution

If:

$$g \nmid b$$

then the congruence has no solution.

Indeed, every expression of the form:

$$ax - mk$$

is divisible by g , whereas b is not divisible by g , leading to a contradiction.

Therefore, the linear congruence equation:

$$ax \equiv b \pmod{m}$$

has solutions if and only if:

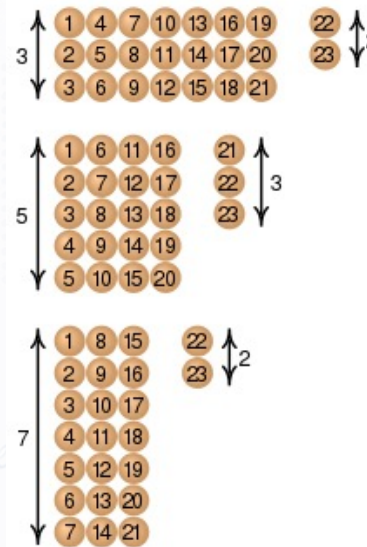
$$\gcd(a, m) \mid b$$

CHINESE REMAINDER THEOREM

The Chinese Remainder Theorem (CRT) is an important result in number theory used to solve systems of linear congruence equations whose moduli are pairwise relatively prime.

In Chinese historical tradition, this theorem is commonly associated with the famous “Han Xin Counting Soldiers” problem. According to legend, the military general Han Xin arranged his soldiers into rows of 3, 5, and 7, observed the remainders, and then determined the exact number of soldiers. Mathematically, this problem corresponds to finding an integer that simultaneously satisfies several congruence conditions.

Today, the Chinese Remainder Theorem has many important applications in large integer computations, modular arithmetic optimization, and cryptography, particularly in RSA cryptosystems.



CHINESE REMAINDER THEOREM

Theorem Statement

Let:

$$m_1, m_2, \dots, m_k$$

be positive integers that are pairwise relatively prime, meaning that:

$$\gcd(m_i, m_j) = 1, \quad i \neq j$$

Consider the system of congruences:

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \dots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

Then the system has a unique solution modulo:

$$M = m_1 m_2 \cdots m_k$$

In other words, all solutions belong to a single residue class modulo M .

CHINESE REMAINDER THEOREM

General Solution Formula

Define:

$$M = m_1 m_2 \cdots m_k$$

and for each:

$$i = 1, 2, \dots, k$$

define:

$$M_i = \frac{M}{m_i}$$

Since the moduli are pairwise relatively prime:

$$\gcd(M_i, m_i) = 1$$

therefore M_i possesses a modular inverse modulo m_i . Let:

$$y_i \equiv M_i^{-1} \pmod{m_i}$$

that is:

$$M_i y_i \equiv 1 \pmod{m_i}$$

Then the solution of the congruence system is:

$$x \equiv a_1 M_1 y_1 + a_2 M_2 y_2 + \cdots + a_k M_k y_k \pmod{M}$$

or equivalently:

$$x \equiv \sum_{i=1}^k a_i M_i y_i \pmod{M}$$

CHINESE REMAINDER THEOREM

Example

An army leaves a remainder of 2 when arranged into rows of 3, a remainder of 3 when arranged into rows of 5, and a remainder of 5 when arranged into rows of 7. Determine the number of soldiers x .

The congruence system is:

$$\begin{cases} x \equiv 2 \pmod{3} \\ x \equiv 3 \pmod{5} \\ x \equiv 5 \pmod{7} \end{cases}$$

Compute:

$$M = 3 \times 5 \times 7 = 105$$

$$M_1 = 35, M_2 = 21, M_3 = 15$$

The modular inverses are:

$$35^{-1} \equiv 2 \pmod{3}$$

$$21^{-1} \equiv 1 \pmod{5}$$

$$15^{-1} \equiv 1 \pmod{7}$$

Applying the CRT formula:

$$x \equiv 2 \times 35 \times 2 + 3 \times 21 \times 1 + 5 \times 15 \times 1 \pmod{105}$$

$$x \equiv 140 + 63 + 75 \equiv 278 \equiv 68 \pmod{105}$$

Therefore:

$$x = 68 + 105k, k \in \mathbb{Z}$$

The smallest positive solution is:

$$x = 68$$

FERMAT'S THEOREM AND ITS EXTENSIONS

Fermat's Little Theorem

If p is a prime number and a is an integer, then:

If p does not divide a , equivalently: $a \not\equiv 0 \pmod{p}$ then:

$$a^p \equiv a \pmod{p}$$

$$a^{p-1} \equiv 1 \pmod{p}$$

This result is known as Fermat's Little Theorem and serves as one of the fundamental principles of modular arithmetic and modern cryptography.

For example:

$$2^5 \equiv 2 \pmod{5}$$

and:

$$2^{5-1} = 2^4 = 16 \equiv 1 \pmod{5}$$

Since:

$$15 \equiv 0 \pmod{5}$$

it follows that:

$$15^4 \equiv 0 \pmod{5}$$

Euler's Theorem (Generalization of Fermat's Theorem)

If $\gcd(a, n) = 1$ then:

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

where: $\varphi(n)$ is Euler's Totient Function, representing the number of positive integers less than n that are relatively prime to n .

Euler's Theorem is a generalization of Fermat's Little Theorem.

In the special case where n is prime:

$$\varphi(n) = n - 1$$

thus Euler's theorem reduces to:

$$a^{n-1} \equiv 1 \pmod{n}$$

which is precisely Fermat's Little Theorem.

For example, when:

$$n = 15$$

we obtain:

$$\varphi(15) = \varphi(3) \times \varphi(5) = 2 \times 4 = 8$$

therefore:

$$7^8 \equiv 1 \pmod{15}$$

and:

$$11^8 \equiv 1 \pmod{15}$$

FERMAT'S THEOREM AND ITS EXTENSIONS

Corollary 1

If $\gcd(c, n) = 1$ and $a \equiv b \pmod{\varphi(n)}$ then:

$$c^a \equiv c^b \pmod{n}$$

Consequently:

$$c^a \bmod n = c^{(b \bmod \varphi(n))} \bmod n$$

This corollary is extremely important because it allows very large exponents to be reduced modulo $\varphi(n)$, significantly simplifying modular exponentiation computations in cryptography.

For example, compute:

$$5^{1005} \pmod{14}$$

Since:

$$\varphi(14) = \varphi(7) \times \varphi(2) = 6 \times 1 = 6$$

and:

$$1005 \equiv 3 \pmod{6}$$

we obtain:

$$5^{1005} \equiv 5^3 \pmod{14}$$

Because:

$$5^3 = 125$$

and:

$$125 \equiv 13 \pmod{14}$$

it follows that:

$$5^{1005} \equiv 13 \pmod{14}$$

Corollary 2

If integers e and d satisfy:

$$ed \equiv 1 \pmod{\varphi(n)}$$

then for every integer c relatively prime to n :

$$(c^e)^d \equiv c \pmod{n}$$

or equivalently:

$$c^{ed} \equiv c \pmod{n}$$

Indeed, since:

$$ed \equiv 1 \pmod{\varphi(n)}$$

Corollary 1 immediately implies:

$$c^{ed} \equiv c^1 \equiv c \pmod{n}$$

This result forms the mathematical foundation of public-key cryptographic systems, particularly the RSA cryptosystem.

In RSA:

- e is the public exponent;
- d is the private exponent;

and the relation:

$$ed \equiv 1 \pmod{\varphi(n)}$$

guarantees that encryption and decryption operations are inverse processes of each other.

ALGEBRAIC STRUCTURES IN MODULAR ARITHMETIC

In modular arithmetic, the set of residue classes modulo n is denoted by:

$$\mathbb{Z}/n\mathbb{Z} = \{0, 1, 2, \dots, n-1\}$$

Over this set, addition, subtraction, and multiplication can be performed modulo n . For example:

$$7 + 8 \equiv 5 \pmod{10}$$

$$7 \times 8 \equiv 6 \pmod{10}$$

An important property is that the result of every operation remains inside the same set $\mathbb{Z}/n\mathbb{Z}$. This property is called *closure*.

Modular arithmetic gives rise to three fundamental algebraic structures:

$$\text{Group} \rightarrow \text{Ring} \rightarrow \text{Field}$$

Each structure plays a different role in cryptography.

A *group* describes a set equipped with an operation and inverse elements. For example, the set of invertible elements modulo n :

$$(\mathbb{Z}/n\mathbb{Z})^*$$

forms a multiplicative group.

A *ring* describes a set equipped with two operations, addition and multiplication. The set:

$$\mathbb{Z}/n\mathbb{Z}$$

always forms a ring under modular addition and multiplication.

A *field* is a special type of ring in which every nonzero element possesses a multiplicative inverse. When p is prime:

$$\mathbb{Z}/p\mathbb{Z}$$

forms a finite field.

These algebraic structures determine which operations are valid, when division is possible, and why certain computational problems become mathematically difficult.

For example:

- RSA operates primarily over the ring: $\mathbb{Z}/n\mathbb{Z}$
- Diffie–Hellman operates over the multiplicative group: $(\mathbb{Z}/p\mathbb{Z})^*$
- ECC uses the Abelian group formed by points on an elliptic curve.

Thus, modular arithmetic is not simply remainder computation; it defines structured computational spaces governed by algebraic laws that serve as the mathematical foundation of modern cryptography.

GROUP

A group is a set: G equipped with a binary operation: $*$ satisfying the following four axioms.

1. Closure

- For all $a, b \in G$ we have: $a * b \in G$
- meaning that the result of the operation always remains inside the set.

2. Associativity

$$(a * b) * c = a * (b * c)$$

- This property allows regrouping of operations without changing the result.

3. Identity Element

- There exists an element: $e \in G$ such that: $a * e = e * a = a$ for every: $a \in G$

4. Inverse Element

- For every $a \in G$ there exists $a^{-1} \in G$ such that: $a * a^{-1} = a^{-1} * a = e$
- If, in addition:

$$a * b = b * a$$

then the group is called an *Abelian group* (or commutative group).

For example:

$$(\mathbb{Z}, +)$$

is an Abelian group under addition.

Similarly:

$$(\mathbb{Z}/n\mathbb{Z}, +)$$

is also an Abelian group.

Groups form the mathematical foundation of many modern cryptographic systems. In Diffie–Hellman and RSA, modular exponentiation is performed over finite groups. In ECC, points on elliptic curves form Abelian groups.

The security of many cryptographic systems relies on computationally difficult problems defined over groups, such as:

- the Discrete Logarithm Problem (DLP);
- the Diffie–Hellman Problem;
- the Elliptic Curve Discrete Logarithm Problem (ECDLP).

RING

A ring is a set R equipped with two operations: “+” and “ $\cdot$ ” satisfying the following properties.

1. Under addition:

$$(R, +)$$

forms an Abelian group. Therefore:

- addition is closed;
- associative;
- commutative;
- contains an additive identity 0;
- every element has an additive inverse.

2. Under multiplication, the ring satisfies:

- Closure
- Associativity

$$\begin{aligned} a, b \in R &\Rightarrow ab \in R \\ (ab)c &= a(bc) \end{aligned}$$

3. Distributive Laws

$$\begin{aligned} a(b + c) &= ab + ac \\ (a + b)c &= ac + bc \end{aligned}$$

The set of integers: $(\mathbb{Z}, +, \cdot)$ is a ring.

Likewise: $\mathbb{Z}/n\mathbb{Z}$ forms a modular ring.

Rings directly model modular arithmetic operations used in cryptography. RSA operates primarily over the ring:

$$\mathbb{Z}/n\mathbb{Z}$$

Modern post-quantum cryptographic systems such as:

- Ring-LWE,
- NTRU,

are constructed using polynomial rings modulo certain ideals.

FIELD

A field is a special type of ring in which every nonzero element possesses a multiplicative inverse. Consequently, within a field it is possible to perform:

- addition,
- subtraction,
- multiplication,
- division (except division by zero).

A field: F must satisfy:

1. Additive Abelian Group $(F, +)$ forms an Abelian group.

2. Multiplicative Abelian Group

$$(F \setminus \{0\}, \cdot)$$

forms an Abelian group.

This guarantees that every nonzero element has a multiplicative inverse.

If p is a prime number, then: $\mathbb{Z}/p\mathbb{Z}$ forms a finite field.

For example:

$$\mathbb{Z}/5\mathbb{Z} = \{0, 1, 2, 3, 4\}$$

and:

$$2^{-1} \equiv 3 \pmod{5}$$

because:

$$2 \times 3 = 6 \equiv 1 \pmod{5}$$

Finite fields are among the most important mathematical structures in modern cryptography.

AES operates over the finite field:

$$GF(2^8)$$

ECC operates over finite fields modulo prime numbers or binary fields.

Many cryptographic algorithms, error-correcting codes, and post-quantum cryptographic systems are fundamentally based on finite field algebra.

FINITE FIELDS

In modern cryptography, most computations are not performed over the set of real numbers or ordinary integers, but rather over finite algebraic structures called finite fields.

Finite fields provide a mathematical environment in which the operations of addition, subtraction, multiplication, and division are all well-defined and always produce results that remain within the same finite space.

A finite field contains a finite number of elements and is generally denoted by:

$$GF(q) \text{ or } \mathbb{F}_q$$

where $q = p^n$

with p being a prime number and $n \geq 1$.

This means that every finite field has a number of elements equal to a power of a prime number.

Prime Field $\mathbb{F}_p$

The simplest finite field is:

$$\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z} = \{0, 1, 2, \dots, p-1\}$$

where p is a prime number.

All arithmetic operations are performed modulo p .

For example, in $\mathbb{F}_7$

we have:

$$5 + 6 \equiv 4 \pmod{7}$$

$$3 \times 5 \equiv 1 \pmod{7}$$

Since $15 \equiv 1 \pmod{7}$ it follows that $3^{-1} \equiv 5 \pmod{7}$

The most important property of a field is that every nonzero element possesses a multiplicative inverse.

This property holds only when the modulus is a prime number.

For example: $\mathbb{Z}/8\mathbb{Z}$ is not a field because $2x \equiv 1 \pmod{8}$

has no solution, meaning that 2 does not have a multiplicative inverse modulo 8.

Set of Nonzero Elements

The set of nonzero elements:

$$\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\} = \{1, 2, \dots, p-1\}$$

plays a particularly important role because every element in this set has a multiplicative inverse modulo p .

This set serves as the mathematical foundation for constructing:

- generators,
- cyclic groups,
- the discrete logarithm problem,
- Diffie–Hellman,
- ElGamal,
- DSA,
- and Elliptic Curve Cryptography (ECC).

GENERATORS – PRIMITIVE ROOTS

In the finite field $\mathbb{F}_p$ where p is a prime number, the set of nonzero elements:

$$\mathbb{F}_p^* = \{1, 2, \dots, p-1\}$$

forms a finite multiplicative group under multiplication modulo p .

One of the most important properties of this group is the existence of special elements capable of “generating” the entire group through modular exponentiation. These elements are called:

generators,
or primitive roots modulo p .

An element:

$$g \in \mathbb{F}_p^*$$

is called a generator if every nonzero element of the field can be represented as a power of g :

$$\mathbb{F}_p^* = \{g^0 \bmod p, g^1 \bmod p, g^2 \bmod p, \dots, g^{p-2} \bmod p\}$$

In other words, by repeatedly multiplying g by itself modulo p , we eventually obtain every nonzero element of the field exactly once before the sequence begins to repeat cyclically.

GENERATORS – PRIMITIVE ROOTS

Example :

For a modulus p , the elements of $\mathbb{Z}/p\mathbb{Z}$ are the residue classes of: $\{1, 2, \dots, p-1\}$ that is, the set of integers relatively prime to p .

Case:

$p=11, \{1, 2, \dots, 10\}$

$m : m^k \pmod{11}$

1 :	1
2 :	2, 4, 8, 5, 10, 9, 7, 3, 6, 1
3 :	3, 9, 5, 4, 1
4 :	4, 5, 9, 3, 1,
5 :	5, 3, 4, 9, 1,
6 :	6, 3, 7, 9, 10, 5, 8, 4, 2, 1
7 :	7, 5, 2, 3, 10, 4, 6, 9, 8, 1
8 :	8, 9, 6, 4, 10, 3, 2, 5, 7, 1 (*)
9 :	9, 4, 3, 5, 1
10 :	10, 1

2, 6, 7, 8 are primitive roots modulo 11,

$p=14, \{1, 3, 5, 9, 11, 13\}$

$m : m^k \pmod{14}$

1 :	1
3 :	3, 9, 13, 11, 5, 1
5 :	5, 11, 13, 9, 3, 1
9 :	9, 11, 1
11 :	11, 9, 1
13 :	13, 1

3 and 5 are primitive roots modulo 14.

GENERATORS – PRIMITIVE ROOTS

- If g is a generator (primitive root) of the group $\mathbb{F}_p^*$ and $\gcd(b, p-1) = 1$ then g^b is also a generator (primitive root) of the group $\mathbb{F}_p^*$

Therefore, the number of generators of the group $\mathbb{F}_p^*$ is exactly equal to the number of integers relatively prime to $(p-1)$, namely: $\phi(p-1)$ where ϕ denotes Euler's Totient Function.

- Example over $\mathbb{F}_{11}^*$ We have: $|\mathbb{F}_{11}^*| = 10$, The number of generators of $\mathbb{F}_{11}^*$ is therefore:
$$\phi(11-1) = \phi(10) = \phi(2 \cdot 5) = \phi(2) \cdot \phi(5) = (2-1)(5-1) = 4$$

The element 2 is a generator of $\mathbb{F}_{11}^*$. Since $\gcd(7, 10) = 1$ it follows that:

$$2^7 \equiv 7 \pmod{11}$$

and therefore: 7 is also a generator of $\mathbb{F}_{11}^*$

Discrete Logarithm Problem (DLP)

Let $g \in \mathbb{F}_{11}^*$ be a generator. Then every element $h \in \mathbb{F}_{11}^*$ can be represented as a power of g :

$$h \equiv g^x \pmod{p}$$

for some integer x .

The problem of determining the exponent x from the equation:

$$h \equiv g^x \pmod{p}$$

is called the:

Discrete Logarithm Problem (DLP).

At present, no efficient polynomial-time algorithm is known for solving this problem in sufficiently large groups.

While the computation of $g^x \pmod{p}$ can be performed efficiently using Fast Modular Exponentiation algorithms, the inverse problem is believed to be computationally hard.

This computational asymmetry forms the security foundation of many modern public-key cryptosystems, including:

- Diffie-Hellman,
- ElGamal,
- DSA,
- and Elliptic Curve Cryptography (ECC).

DISCRETE LOGARITHM PROBLEM

After establishing that the multiplicative group:

$$\mathbb{F}_p^*$$

can be generated by a generator g , every nonzero element can be represented as a power of g . That is, for every:

$$b \in \mathbb{F}_p^*$$

there always exists an integer x such that:

$$g^x \equiv b \pmod{p}$$

The number x is called the discrete logarithm of b to the base g , denoted by:

$$x = \log_g b$$

in the group:

$$\mathbb{F}_p^*$$

The crucial point is that the forward computational direction is easy, while the inverse direction is computationally difficult.

Given g , x , and p , we can efficiently compute:

$$b \equiv g^x \pmod{p}$$

using fast modular exponentiation algorithms.

However, if only g , b , and p are known, recovering x from:

$$g^x \equiv b \pmod{p}$$

is considered a computationally hard problem when p is sufficiently large and properly chosen.

DISCRETE LOGARITHM PROBLEM

Example:

Let $p = 101$ and let 2 be a generator of $\mathbb{F}_{101}^*$

If $2^x \equiv 5 \pmod{101}$ then the solution is $x = 24$ since $2^{24} \equiv 5 \pmod{101}$

Similarly, if $2^x \equiv 6 \pmod{101}$ then $x = 70$ because $2^{70} \equiv 6 \pmod{101}$

For small numbers, we may determine x by exhaustively testing possible values. However, when p has hundreds or thousands of bits, brute-force search becomes computationally infeasible.

An important observation is that the difficulty of the discrete logarithm problem depends strongly on the structure of the group. If the group order:

$$p - 1$$

factorizes into many small prime factors, the problem may be solved more efficiently using algorithms such as the Pohlig–Hellman algorithm.

Therefore, in practical cryptography, p is usually chosen such that:

$$\mathbb{F}_p^*$$

contains a large prime-order subgroup.

The discrete logarithm problem forms the security foundation of many public-key cryptosystems. Systems such as Diffie–Hellman, ElGamal, and DSA all rely on the assumption that modular exponentiation is computationally easy, while the inverse problem — recovering the exponent from the modular result — is computationally hard.

Easy: $x \rightarrow g^x \pmod{p}$

Hard: $g^x \pmod{p} \rightarrow x$

This computational asymmetry is precisely what provides the security foundation for cryptographic protocols based on the discrete logarithm problem.

CRYPTOGRAPHIC SIGNIFICANCE

Most public-key cryptosystems do not operate directly on random numbers, but instead work within a multiplicative group modulo p .

We require:

- an operation that is computationally easy (computing $g^a \bmod p$ is efficient),
- but computationally difficult to reverse (recovering a from $g^a \bmod p$ is extremely hard).

This property holds only when g is a primitive root (generator), because in that case:

- every possible value from 1 to $p - 1$ can be represented as:
- $g^k \bmod p$
- there are no short cycles, preventing premature repetition of values,
- the generated values are distributed uniformly and exhibit strong pseudorandom behavior.

Therefore, a primitive root provides the strongest possible key space.

If g is not a primitive root, then:

$$\{g^k \bmod p\}$$

generates only a small subset of the group, making the system more vulnerable to brute-force attacks or reduced discrete logarithm attacks.

FINITE FIELD $GF(2^n)$

In the prime field:

$$GF(2) = \{0, 1\}$$

there are only two elements, which is insufficient for constructing complex cryptographic systems and digital data-processing mechanisms.

To create larger algebraic spaces while still preserving all field properties, extension fields are constructed.

The field $GF(2^n)$ is built from polynomials whose coefficients belong to:

$$GF(2) = \{0, 1\}$$

Polynomial Arithmetic over **$GF(2)$** :

Every polynomial in $GF(2)[x]$ has the form:

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

where: $a_i \in GF(2)$

Since each coefficient can only be either 0 or 1, the number of distinct polynomials of degree at most n is: 2^{n+1} .

In $GF(2)$ addition corresponds to the XOR operation: $1 + 1 = 0$. Therefore, identical terms cancel each other out.

For example:

$$(x^4 + x^2 + x)(x^2 + x) = x^6 + x^5 + x^4 + x^3 + x^3 + x^2 = x^6 + x^5 + x^4 + x^2$$

Polynomial addition in: $GF(2)$ is therefore equivalent to bitwise XOR operations in computer systems.

Construction of $GF(2^n)$

The fundamental idea behind constructing $GF(2^n)$ is the selection of an irreducible polynomial to serve as the polynomial modulus, analogous to using a prime modulus in $GF(p)$.

This irreducible polynomial defines the arithmetic structure of the extension field and guarantees that every nonzero element possesses a multiplicative inverse, thereby preserving the field properties.

CONSTRUCTION OF THE FIELD $GF(2^n)$

To construct: $GF(2^n)$ we choose an irreducible polynomial $P(x)$ of degree n over $GF(2)$ and consider the quotient ring:

$$GF(2)[x]/P(x)$$

Every element of the field is represented by a polynomial of degree less than n .

For example, let:

$$P(x) = x^3 + x + 1$$

which is an irreducible polynomial over $GF(2)$. Then:

$$GF(2^3) = GF(2)[x]/(x^3 + x + 1)$$

contains exactly $2^3 = 8$ elements:

$$\{0, 1, x, x + 1, x^2, x^2 + 1, x^2 + x, x^2 + x + 1\},$$

Within this field:

$$\begin{aligned} x^3 + x + 1 &= 0 \\ x^3 &= x + 1 \end{aligned}$$

Therefore:

As a result, every higher-degree power can be reduced to a polynomial of degree less than 3.

For example:

$$x^4 = x \cdot x^3 = x(x + 1) = x^2 + x$$

and:

$$\begin{aligned} (x^2 + x + 1)(x + 1) &= x^3 + 1 \\ &\equiv (x + 1) + 1 = x \end{aligned}$$

in:

$$GF(2^3)$$

MULTIPLICATIVE STRUCTURE OF $GF(2^n)$

The set of nonzero elements of $GF(2^n)$ is denoted by:

$$GF(2^n)^*$$

and forms a finite multiplicative group containing: $2^n - 1$ elements.

This group is also cyclic, meaning that there exists a generator: $g \in GF(2^n)^*$ such that every nonzero element can be represented in the form: g^k

Example

In $GF(2^3)$ let $g = x$ then: $g^2 = x^2$, $g^3 = x + 1$, $g^4 = x^2 + x$, $g^5 = x^2 + x + 1$, $g^6 = x^2 + 1$, $g^7 = 1$

All seven nonzero elements appear exactly once, therefore: x is a generator of $GF(2^3)^*$

This demonstrates that $GF(2^n)^*$ has a structure analogous to the multiplicative group $GF(p)^*$ discussed previously.

Binary Representation

One of the most important advantages of:

$$GF(2^n)$$

is that every element can be represented directly as a binary bit string.

For example:

$$\begin{aligned} & x^4 + x^2 + 1 \\ & 1x^4 + 0x^3 + 1x^2 + 0x + 1 \end{aligned}$$

can be written as:

which corresponds to the binary representation: 10101

Therefore:

- addition corresponds to XOR operations,
- multiplication corresponds to polynomial multiplication modulo an irreducible polynomial,
- and all arithmetic operations can be implemented efficiently using digital logic circuits.

Applications in Cryptography

For this reason $GF(2^n)$ is widely used in: AES, CRC, Reed–Solomon codes, BCH codes, storage systems, and communication systems.

In particular, the Advanced Encryption Standard algorithm operates directly over:

$$GF(2^8)$$

where each byte is interpreted as an element of the finite field.

ELLIPTIC CURVES

Elliptic Curves and Group Structure

An elliptic curve is a special algebraic object defined by a nonsingular cubic equation together with a distinguished point called the Point at Infinity.

Over the field of real numbers: $\mathbb{R}$

the general Weierstrass equation is written as:

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$$

The term *elliptic curve* does not originate from the geometric shape of the curve, but rather from elliptic integrals that arose in the classical calculus problem of computing the arc length of an ellipse.

1. Point at Infinity and Projective Space

In order for the set of points on the curve to form a complete algebraic group, an additional special point must be introduced, denoted by $\mathcal{O}$ called the Point at Infinity.

In projective geometry, this point is represented in the projective plane:

$$\mathbb{P}^2(\mathbb{R})$$

using homogeneous coordinates $(0:1:0)$. The point $\mathcal{O}$ acts as the identity element for the point-addition operation on the elliptic curve.

2. Group Law on Elliptic Curves

Let three points P , Q , R lie on the same straight line intersecting the elliptic curve.

According to Bézout's Theorem, a line intersects a cubic curve at exactly three intersection points (counting multiplicities).

We therefore define $P + Q + R = \mathcal{O}$ which implies: $P + Q = -R$

where the inverse point of R is obtained by reflection across the x -axis.

If $P = (x, y)$ then the inverse of P is $-P = (x, -y)$ and consequently $P + (-P) = \mathcal{O}$

3. Abelian Group Structure

The set of all points on the elliptic curve together with the Point at Infinity $E(\mathbb{R}) \cup \{\mathcal{O}\}$ forms an Abelian group under the point-addition operation.

The group satisfies the following properties:

Closure $P, Q \in E \Rightarrow P + Q \in E$

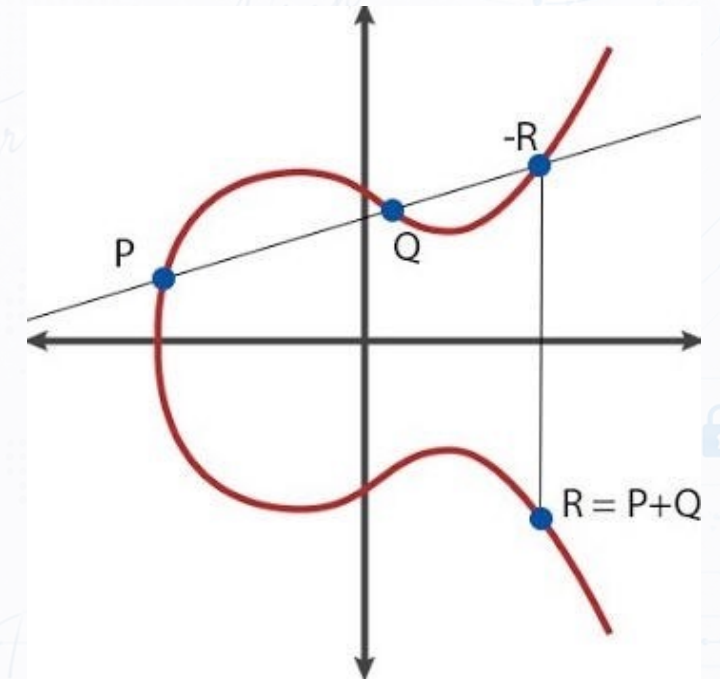
Identity Element $P + \mathcal{O} = P$

Inverse Element $P + (-P) = \mathcal{O}$

Commutativity $P + Q = Q + P$

Associativity $(P + Q) + R = P + (Q + R)$

Among these axioms, associativity is the most difficult property to prove and typically requires advanced tools from algebraic geometry and intersection theory.



ELLIPTIC CURVES

Point Addition and Point Doubling

When the characteristic of the field satisfies:

$$\text{char}(K) \neq 2, 3$$

the general Weierstrass equation can be transformed into the simplified form:

$$y^2 = x^3 + ax + b$$

For the curve to be nonsingular, the discriminant must be nonzero:

$$\Delta = -16(4a^3 + 27b^2) \neq 0$$

This condition guarantees that the curve does not contain singular points such as:

cusps,
or self-intersections.

1. Addition of Two Distinct Points

Let two distinct points be given:

$$P = (x_1, y_1), Q = (x_2, y_2)$$

The straight line passing through P and Q has slope:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

This line intersects the elliptic curve at a third point R' . We then define:

$$P + Q = -R'$$

2. Point Doubling

When $P = Q$ we consider the tangent line to the curve at the point P . The slope of the tangent line is:

$$\lambda = \frac{3x_1^2 + a}{2y_1}$$

After determining λ the coordinates of $R = P + Q = (x_3, y_3)$ are computed as:

$$\begin{aligned} x_3 &= \lambda^2 - x_1 - x_2 \\ y_3 &= \lambda(x_1 - x_3) - y_1 \end{aligned}$$

3. Arithmetic over Finite Fields

In cryptography, all arithmetic operations are performed over finite fields modulo p .

Therefore, division $\frac{A}{B}$ is not performed directly, but instead rewritten as $A \cdot B^{-1} \pmod{p}$ where B^{-1} is the modular inverse of B , satisfying:

$$BB^{-1} \equiv 1 \pmod{p}$$

The modular inverse is typically computed using the Extended Euclidean Algorithm.

ELLIPTIC CURVES

Elliptic Curves over Finite Fields

In Elliptic Curve Cryptography (ECC), elliptic curves are not constructed over $\mathbb{R}$ but rather over the finite field $\mathbb{F}_p$ with $p > 3$

The set of points on the curve is then defined by:

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 \mid y^2 \equiv x^3 + ax + b \pmod{p}\} \cup \{\mathcal{O}\}$$

Unlike the case over the real numbers, the points of the curve no longer form a continuous geometric curve, but instead form a finite set of discrete points in modular space.

However, the entire algebraic structure of point addition remains preserved.

1. Example over $\mathbb{F}_5$

Consider the elliptic curve:

We have:

$$E: y^2 \equiv x^3 + x + 1 \pmod{5}$$

$$a = 1, b = 1$$

Checking the discriminant:

$$\begin{aligned}\Delta &= -16(4a^3 + 27b^2) \\ &= -16(4 + 27) \\ &= -496 \equiv 4 \pmod{5} \neq 0\end{aligned}$$

Therefore, the curve is nonsingular.

Enumerating all $x \in \{0, 1, 2, 3, 4\}$

gives the following points: (0, 1); (0, 4); (2, 1); (2, 4); (3, 1); (3, 4); (4, 2); (4, 3)

together with the Point at Infinity $\mathcal{O}$

Hence:

$$\#E(\mathbb{F}_5) = 9$$

2. Hasse's Theorem

The number of points on a finite elliptic curve is bounded by Hasse's Theorem:

$$|\#E(\mathbb{F}_p) - (p + 1)| \leq 2\sqrt{p}$$

This theorem plays a central role in ECC theory because it provides an estimate for the size of the elliptic-curve point group.

ELLIPTIC CURVES

Cyclic Subgroups and the Discrete Logarithm Problem

The fundamental operation in Elliptic Curve Cryptography (ECC) is scalar multiplication $Q = kP$ which means adding the point P to itself k times:

$$kP = \underbrace{(P + P + \cdots + P)}_{k \text{ times}}$$

1. Optimization of Scalar Multiplication

In practice, this operation is not performed through naive repeated addition, but instead uses optimized algorithms such as:

- Double-and-Add,
- Window Method,
- Montgomery Ladder.

These algorithms reduce the computational complexity to:

$$O(\log k)$$

2. Cyclic Subgroups

Given $P \in E(\mathbb{F}_p)$ the set:

$$\langle P \rangle = \{\mathcal{O}, P, 2P, 3P, \dots\}$$

forms a cyclic subgroup generated by P .

The smallest positive integer n satisfying $nP = \mathcal{O}$ is called the order of the point P .

3. Cofactor and Security Requirements

In practical ECC systems, one typically selects:

- a subgroup of very large prime order,
- together with a small cofactor,

in order to resist attacks such as the Pohlig–Hellman attack.

If $\#E(\mathbb{F}_p) = hn$ then:

- n is the large prime order of the generated subgroup,
- h is called the cofactor.

SUMMARY

The computations used in modern cryptography typically involve extremely high mathematical and computational complexity. In addition to handling large-scale arithmetic operations, modern cryptographic systems also operate on various abstract algebraic structures such as finite fields, cyclic groups, discrete logarithms, polynomial arithmetic modulo irreducible polynomials, and points on elliptic curves. These mathematical objects are usually defined over very large finite spaces in order to guarantee computational security.

Due to the size of the data and the complexity of the underlying algorithms, it is practically impossible to perform cryptographic computations manually. Consequently, the study, implementation, and verification of modern cryptographic systems rely heavily on computer-based computation and specialized mathematical software.

In practice, developing complete computational libraries and cryptographic computation systems from scratch generally requires deep expertise in mathematics, algorithms, and high-performance programming. Therefore, computer algebra systems and specialized cryptographic software tools play a critically important role in the study, research, and deployment of modern secure systems.